PROJECT: DEBUGGING CODE

datalab

In today's fast-paced, data-driven world, companies rely heavily on accurate analyses of sales data to make informed decisions. However, the quality of the insights derived from such analyses depend significantly on the integrity of the underlying code.

You have been given some starting code with two functions: one that extracts and flattens JSON data into a structured format and the other that transforms electricity sales data by cleaning, filtering, and extracting relevant features. The company plans to use your revised code to improve the accuracy of sales analytics.

Your task is to identify potential errors in the functions and the underlying data that might result in logic and runtime errors, such as missing values, incorrect data types, or incompatible values (e.g., negatives). Enhance the custom functions provided by implementing exceptions to catch data quality issues and edge cases.

The data is available here ("sales_data_sample.csv") for the analyses. This data has 25 columns, but only two columns are analyzed, namely, `quantity_ordered` and `price_each`. A sample of the data is shown below.

```
# Import libraries
import pandas as pd

# Load data
sales_df = pd.read_csv("data/sales_data_sample.csv")
sales_df.head()
```

	orde...	quantity_or...	p...	order_line_nu...		order_date			
0	10107	30	95.7	2	2871	2/24/2003 0:00	Shipped	1	
1	10121	34	81.35	5	2765.9	5/7/2003 0:00	Shipped	2	
2	10134	-41	94.74	2	3884.34	7/1/2003 0:00	Shipped	3	
3	10145	45		6	3746.7	8/25/2003 0:00	Shipped	3	
4	10159	49	100	14	5205.27	10/10/2003 0:00	Shipped	4	1

Rows: 5 Expand

```
# Identify errors and add exceptions to the `get_quantity_ordered_sum()` function
def get_quantity_ordered_sum(sales_quantity_ordered):
    """Calculates the total sum on the 'quantity_ordered' column.

    Args:
        sales_quantity_ordered (pd.core.series.Series): The pandas Series for the 'quantity_ordered' column.

    Returns:
        total_quantity_ordered (int): The total sum of the 'quantity_ordered' column.
    """

    total_quantity_ordered = 0
    for quantity in sales_quantity_ordered:
        total_quantity_ordered += quantity
    return total_quantity_ordered
```

```python
# Identify errors and add exceptions to the `get_price_each_average()` function
def get_price_each_average(sales_price_each, num_places=2):
    """Calculates the average on the 'price_each' column
       using pandas built in methods and rounds to the desired number of places.

    Args:
        sales_price_each (pd.core.series.Series): The pandas Series for the 'price_each' column.
        num_of_places (int): The number of decimal places to round.

    Returns:
        average_price_each (float): The average of the 'price_each' column.
    """

    total_of_price_each = sales_price_each.sum()
    len_of_price_each = len(sales_price_each)
    average_price_each = round(
        total_of_price_each / len_of_price_each, num_places
    )
    return average_price_each
```

```python
#  Add as many cells as you require
import pandas as pd

def get_quantity_ordered_sum(sales_quantity_ordered):
    """Calculates the total sum on the 'quantity_ordered' column.

    Args:
        sales_quantity_ordered (pd.core.series.Series): The pandas Series for the 'quantity_ordered' column.

    Returns:
        total_quantity_ordered (int): The total sum of the 'quantity_ordered' column.
    """
    total_quantity_ordered = 0  # Initialize total sum variable
    try:
        for quantity in sales_quantity_ordered:
            if quantity < 0:
                # Convert negative values to positive to ensure proper summation
                quantity *= -1
            total_quantity_ordered += quantity  # Accumulate quantity values
    except TypeError:
        # Raise an error if a non-numeric value is encountered
        raise TypeError("Invalid data type in 'quantity_ordered' column.")

    return total_quantity_ordered  # Return the computed total sum


def get_price_each_average(sales_price_each, num_of_places=2):
    """Calculates the average on the 'price_each' column
       using pandas built-in methods and rounds to the desired number of places.

    Args:
        sales_price_each (pd.core.series.Series): The pandas Series for the 'price_each' column.
        num_of_places (int): The number of decimal places to round.

    Returns:
        average_price_each (float): The average of the 'price_each' column.
    """
    try:
        # Compute total sum of 'price_each' column
        total_of_price_each = sales_price_each.sum()
        # Compute length of the 'price_each' column
        len_of_price_each = len(sales_price_each)
        # Compute and round the average
        average_price_each = round(
            total_of_price_each / len_of_price_each, num_of_places
        )
    except TypeError:
        # Handle cases where non-numeric values exist in the column
        sales_price_each = pd.to_numeric(sales_price_each, errors='coerce')  # Convert non-numeric to NaN
        sales_price_each.fillna(sales_price_each.mean())  # Replace NaN with column mean

        # Recalculate total sum and length after handling invalid data
        total_of_price_each = sales_price_each.sum()
        len_of_price_each = len(sales_price_each)

        # Compute and round the average again
        average_price_each = round(
            total_of_price_each / len_of_price_each, num_of_places
        )

    return average_price_each  # Return the computed average price

# Example usage with a DataFrame (assuming 'sales_df' exists)
total_quantity_ordered = get_quantity_ordered_sum(sales_df['quantity_ordered'])  # Compute total quantity ordered
average_price_each = get_price_each_average(sales_df['price_each'])  # Compute average price per item

# Print the results
print(total_quantity_ordered)
print(average_price_each)
```

```
99067
83.63
```